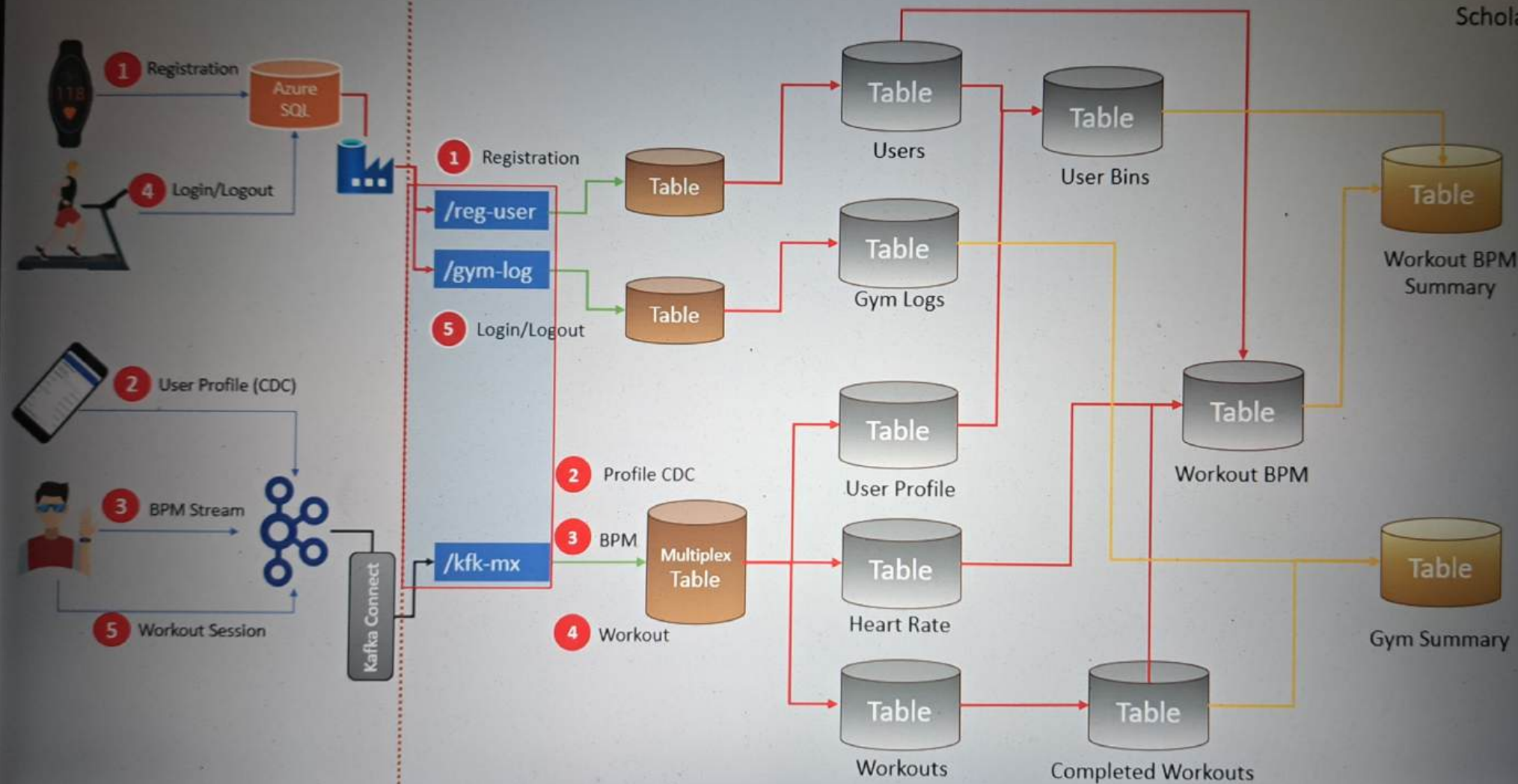
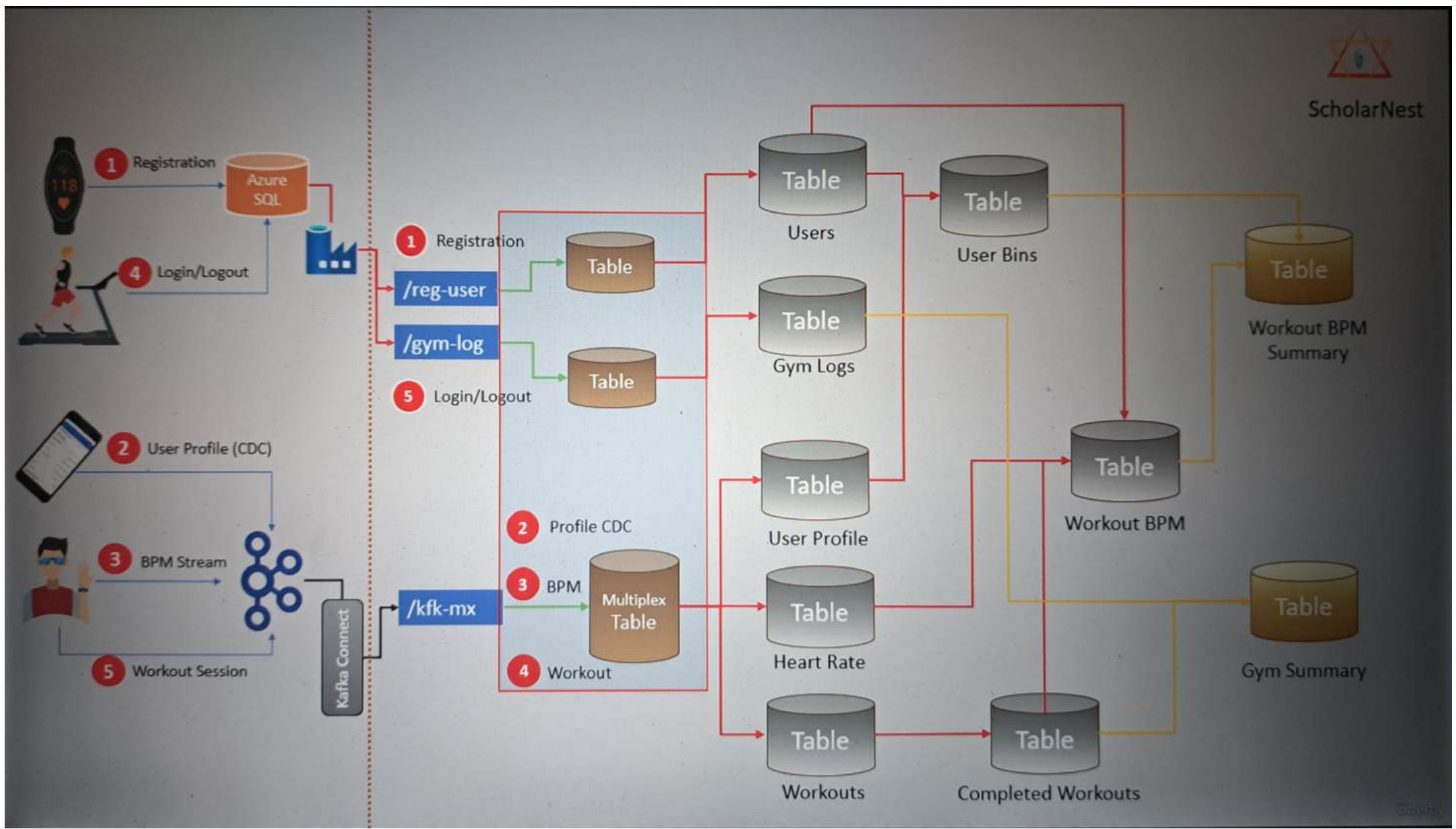
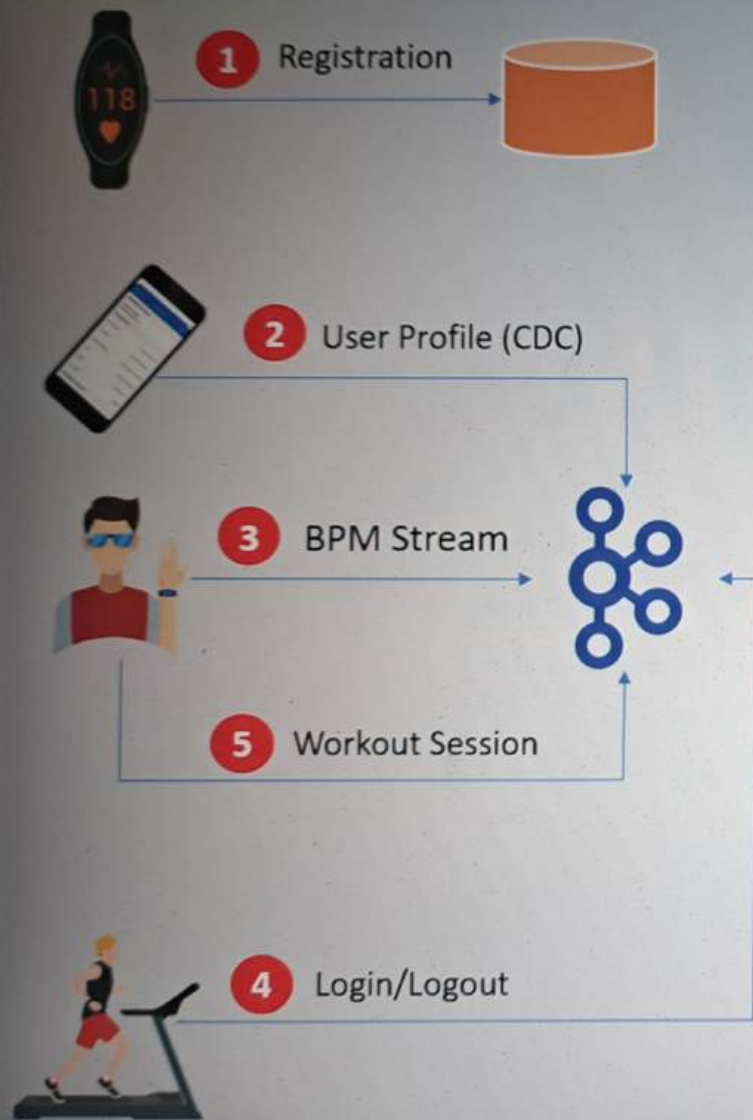








1. Batch Processing Approach
2. Stream Processing Approach

- I. Support batch run at regular intervals
- II. Supports continuous streaming run
- III. Automatic checkpointing
- IV. Automatic restart

+ New

📁 Workspace

🕒 Recents

📊 Data

🔄 Workflows

💻 Compute

SQL

📄 SQL Editor

🔍 Queries

📊 Dashboards

🔔 Alerts

🕒 Query History

🏠 SQL Warehouses

Data Engineering

🔄 Job Runs

📊 Data Ingestion

📊 Delta Live Tables

Machine Learning

04-bronze

🔗 initial-feature

Python ▾

☆

File Edit View Run Help

Last edit was now

Provide feedback

▶ Run all

🔗 Connect ▾

📅 Schedule

Share

📄

Cmd 1

1 %run ./01-config

📄

Cmd 2

```
1 class Bronze():
2     def __init__(self, env):
3         self.Conf = Config()
4         self.landing_zone = self.Conf.base_dir_data + "/raw"
5         self.checkpoint_base = self.Conf.base_dir_checkpoint + "/checkpoints"
6         self.catalog = env
7         self.db_name = self.Conf.db_name
8         spark.sql(f"USE {self.catalog}.{self.db_name}")
9
10    def consume_user_registration(self, once=True, processing_time="5 seconds"):
11        from pyspark.sql import functions as F
12        schema = "user_id long, device_id long, mac_address string, registration_timestamp double"
13
14        df_stream = (spark.readStream
15                      .format("cloudFiles")
16                      .schema(schema)
17                      .option("maxFilesPerTrigger", 1)
18                      .option("cloudFiles.format", "csv")
19                      .option("header", "true")
20                      .load(self.landing_zone + "(registered_users.bz)"))
```

New

Workspace

Recents

Data

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

04-bronze initial-feature Python

File Edit View Run Help Last edit was now Provide feedback

Run all

Connect

Schedule

Share

```
1 class Bronze():
2     def __init__(self, env):
3         self.Conf = Config()
4         self.landing_zone = self.Conf.base_dir_data + "/raw"
5         self.checkpoint_base = self.Conf.base_dir_checkpoint + "/checkpoints"
6         self.catalog = env
7         self.db_name = self.Conf.db_name
8         spark.sql(f"USE {self.catalog}.{self.db_name}")
9
10    def consume_user_registration(self, once=True, processing_time="5 seconds"):
11        from pyspark.sql import functions as F
12        schema = "user_id long, device_id long, mac_address string, registration_timestamp double"
13
14        df_stream = (spark.readStream
15                      .format("cloudFiles")
16                      .schema(schema)
17                      .option("maxFilesPerTrigger", 1)
18                      .option("cloudFiles.format", "csv")
19                      .option("header", "true")
20                      .load(self.landing_zone + "/registered_users_bz")
21                      .withColumn("load_time", F.current_timestamp())
22                      .withColumn("source_file", F.input_file_name())
23                      )
```

Uses append mode because bronze layer is expected to insert only from source

04-bronze - Databricks

adb-3007342644227763.3.azure.databricks.net/?o=3007342644227763#notebook/2761304389330991/command/2761304389330993

Microsoft Azure databricks Search data, notebooks, recents, and more... CTRL + P sbit-ws prashant@scholarnest.com

New

Workspace

Recents

Data

Workflows

Compute

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

04-bronze initial-feature Python

Last edit was 1 minute ago Provide feedback

Run all Connect Schedule Share

```
22         .writeColumn(source_file, F.input_file_name())
23     )
24
25     # Use append mode because bronze layer is expected to insert only from source
26     stream_writer = df_stream.writeStream \
27         .format("delta") \
28         .option("checkpointLocation", self.checkpoint_base + "/registered_users_bz") \
29         .outputMode("append") \
30         .queryName("registered_users_bz_ingestion_stream")
31
32     spark.sparkContext.setLocalProperty("spark.scheduler.pool", "bronze_p2")
33
34     if once == True:
35         return stream_writer.trigger(availableNow=True).toTable(f"{self.catalog}.{self.db_name}.registered_users_bz")
36     else:
37         return stream_writer.trigger(processingTime=processing_time).toTable(f"{self.catalog}.{self.db_name}.registered_users_bz")
38
39     def consume_gym_logins(self, once=True, processing_time="5 seconds"):
40         from pyspark.sql import functions as F
41         schema = "mac_address string, gym bigint, login double, logout double"
42
43         df_stream = (spark.readStream
44             .format("cloudFiles")
45             .schema(schema)
46             .option("maxFilesPerTrigger", 1)
47             .option("cloudFiles.format", "csv")
```


New

Workspace

Recents

Data

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

04-bronze initial-feature Python

File Edit View Run Help Last edit was 1 minute ago Provide feedback

Run all

Connect

Schedule

Share

registered_users_bz")

```
38
39 def consume_gym_logins(self, once=True, processing_time="5 seconds"):
40     from pyspark.sql import functions as F
41     schema = "mac_address string, gym bigint, login double, logout double"
42
43     df_stream = (spark.readStream
44                  .format("cloudFiles")
45                  .schema(schema)
46                  .option("maxFilesPerTrigger", 1)
47                  .option("cloudFiles.format", "csv")
48                  .option("header", "true")
49                  .load(self.landing_zone + "/gym_logins_bz")
50                  .withColumn("load_time", F.current_timestamp())
51                  .withColumn("source_file", F.input_file_name())
52                  )
53
54     # Use append mode because bronze layer is expected to insert only from source
55     stream_writer = df_stream.writeStream \
56         .format("delta") \
57         .option("checkpointLocation", self.checkpoint_base + "/gym_logins_bz") \
58         .outputMode("append") \
59         .queryName("gym_logins_bz_ingestion_stream")
60
61     spark.sparkContext.setLocalProperty("spark.scheduler.pool", "bronze_p2")
62
63     if once == True:
```

☆

File Edit View Run Help Last edit was 1 minute ago Provide feedback

▶ Run all

- Connect ▾

 Schedule

```

59         .queryName("gym_logins_bz_ingestion_stream")
60
61     spark.sparkContext.setLocalProperty("spark.scheduler.pool", "bronze_p2")
62
63     if once == True:
64         return stream_writer.trigger(availableNow=True).toTable(f"{self.catalog}.{self.db_name}.gym_logins_bz")
65     else:
66         return stream_writer.trigger(processingTime=processing_time).toTable(f"{self.catalog}.{self.db_name}.gym_logins_bz")
67
68
69     def consume_kafka_multiplex(self, once=True, processing_time="5 seconds"):
70         from pyspark.sql import functions as F
71         schema = "key string, value string, topic string, partition bigint, offset bigint, timestamp bigint"
72         df_date_lookup = spark.table(f"{self.catalog}.{self.db_name}.date_lookup").select("date", "week_part")
73
74         df_stream = (spark.readStream
75             .format("cloudFiles")
76             .schema(schema)
77             .option("maxFilesPerTrigger", 1)
78             .option("cloudFiles.format", "json")
79             .load(self.landing_zone + "/kafka_multiplex_bz")
80             .withColumn("load_time", F.current_timestamp())
81             .withColumn("source_file", F.input_file_name())
82             .join(F.broadcast(df_date_lookup),
83                 [F.to_date((F.col("timestamp")/1000).cast("timestamp")) = F.col("date")],
84                 "true")

```


04-bronze - Databricks

adb-3007342644227763.3.azure.databricks.net/?o=3007342644227763#notebook/2761304389330991/command/2761304389330993

Microsoft Azure databricks Search data, notebooks, recents, and more... CTRL + P sbit-ws prashant@scholarnest.com

New

WorkspaceRecentsDataWorkflowsComputeSQL EditorQueriesDashboardsAlertsQuery HistorySQL WarehousesData EngineeringJob RunsData IngestionDelta Live TablesMachine Learning

04-bronze initial-feature Python ☆

File Edit View Run Help Last edit was 1 minute ago Provide feedback

Run all Connect Schedule Share

```
67
68
69 def consume_kafka_multiplex(self, once=True, processing_time="5 seconds"):
70     from pyspark.sql import functions as F
71     schema = "key string, value string, topic string, partition bigint, offset bigint, timestamp bigint"
72     df_date_lookup = spark.table(f"{self.catalog}.{self.db_name}.date_lookup").select("date", "week_part")
73
74     df_stream = (spark.readStream
75                  .format("cloudFiles")
76                  .schema(schema)
77                  .option("maxFilesPerTrigger", 1)
78                  .option("cloudFiles.format", "json")
79                  .load(self.landing_zone + "/kafka_multiplex_bz")
80                  .withColumn("load_time", F.current_timestamp())
81                  .withColumn("source_file", F.input_file_name())
82                  .join(F.broadcast(df_date_lookup),
83                       [F.to_date((F.col("timestamp")/1000).cast("timestamp")) == F.col("date")],
84                       "left")
85                  )
86
87     # Use append mode because bronze layer is expected to insert only from source
88     stream_writer = df_stream.writeStream \
89                     .format("delta") \
90                     .option("checkpointLocation", self.checkpoint_base + "/kafka_multiplex_bz") \
91                     .outputMode("append") \
92                     .queryName("kafka_multiplex_bz_ingestion_stream")
93
```

New

Workspace

Recents

Data

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

04-bronze

initial-feature

Python



File Edit View Run Help Last edit was 1 minute ago Provide feedback

Run all

Connect

Schedule

Share

```
89         .format("delta") \
90         .option("checkpointLocation", self.checkpoint_base + "/kafka_multiplex_bz") \
91         .outputMode("append") \
92         .queryName("kafka_multiplex_bz_ingestion_stream")
93
94     spark.sparkContext.setLocalProperty("spark.scheduler.pool", "bronze_pl")
95
96     if once == True:
97         return stream_writer.trigger(availableNow=True).toTable(f"{self.catalog}.{self.db_name}.kafka_multiplex_bz")
98     else:
99         return stream_writer.trigger(processingTime=processing_time).toTable(f"{self.catalog}.{self.db_name}.kafka_multiplex_bz")
100
101
102     def consume(self, once=True, processing_time="5 seconds"):
103         import time
104         start = int(time.time())
105         print(f"\nStarting bronze layer consumption ...")
106         self.consume_user_registration(once, processing_time)
107         self.consume_gym_logins(once, processing_time)
108         self.consume_kafka_multiplex(once, processing_time)
109         if once:
110             for stream in spark.streams.active:
111                 stream.awaitTermination()
112         print(f"Completed bronze layer consumption {int(time.time()) - start} seconds")
113
114
```

New

Workspace

Recents

Data

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

04-bronze

initial-feature

Python



File Edit View Run Help Last edit was 1 minute ago Provide feedback

Run all

Connect

Schedule

Share

```
102 def consume(self, once=True, processing_time="5 seconds"):
103     import time
104     start = int(time.time())
105     print(f"\nStarting bronze layer consumption ...")
106     self.consume_user_registration(once, processing_time)
107     self.consume_gym_logins(once, processing_time)
108     self.consume_kafka_multiplex(once, processing_time)
109     if once:
110         for stream in spark.streams.active:
111             stream.awaitTermination()
112     print(f"Completed bronze layer consumption {int(time.time()) - start} seconds")
113
114
115 def assert_count(self, table_name, expected_count, filter="true"):
116     print(f"Validating record counts in {table_name}...", end='')
117     actual_count = spark.read.table(f"{self.catalog}.{self.db_name}.{table_name}").where(filter).count()
118     assert actual_count == expected_count, f"Expected {expected_count:,} records, found {actual_count:,} in {table_name} where {filter}"
119     print(f"Found {actual_count:,} / Expected {expected_count:,} records where {filter}: Success")
120
121 def validate(self, sets):
122     import time
123     start = int(time.time())
124     print(f"\nValidating bronz layer records...")
125     self.assert_count("registered_users_bz", 5 if sets == 1 else 10)
126     self.assert_count("gym_logins_bz", 8 if sets == 1 else 16)
127     self.assert_count("kafka_multiplex_bz", 7 if sets == 1 else 12, "topic: user_info")
```


➕ New

📁 Workspace

🕒 Recents

📊 Data

🔄 Workflows

☁ Compute

SQL

📝 SQL Editor

📄 Queries

📊 Dashboards

🔔 Alerts

🕒 Query History

☁ SQL Warehouses

Data Engineering

🔄 Job Runs

📊 Data Ingestion

🔄 Delta Live Tables

Machine Learning

04-bronze

🔗 initial-feature

Python ▾

☆

File Edit View Run Help [Last edit was 2 minutes ago](#) [Provide feedback](#)

▶ Run all

● Connect ▾

📅 Schedule

Share

```

111         stream.awaitTermination()
112         print(f"Completed bronze layer consumption {int(time.time()) - start} seconds")
113
114
115     def assert_count(self, table_name, expected_count, filter="true"):
116         print(f"Validating record counts in {table_name}...", end='')
117         actual_count = spark.read.table(f"{self.catalog}.{self.db_name}.{table_name}").where(filter).count()
118         assert actual_count == expected_count, f"Expected {expected_count:,} records, found {actual_count:,} in {table_name} where {filter}"
119         print(f"Found {actual_count:,} / Expected {expected_count:,} records where {filter}: Success")
120
121     def validate(self, sets):
122         import time
123         start = int(time.time())
124         print(f"\nValidating bronz layer records...")
125         self.assert_count("registered_users_bz", 5 if sets == 1 else 10)
126         self.assert_count("gym_logins_bz", 8 if sets == 1 else 16)
127         self.assert_count("kafka_multiplex_bz", 7 if sets == 1 else 13, "topic='user_info'")
128         self.assert_count("kafka_multiplex_bz", 16 if sets == 1 else 32, "topic='workout'")
129         self.assert_count("kafka_multiplex_bz", sets * 253801, "topic='bpm'")
130         print(f"Bronze layer validation completed in {int(time.time()) - start} seconds")

```

Shift+Enter to run

Shift+Ctrl+Enter to run selected text